A CONTRASTIVE REPRESENTATION LEARNING MODEL FOR BRAIN TUMOR IDENTIFICATION

A PREPRINT

Department of Computer Science
Aarhus University

Department of Computer Science
Aarhus University

Department of Computer Science
Aarhus University

December 3, 2025

ABSTRACT

This project explores Contrastive Representation Learning (CRL) for brain tumor classification using the BRISC dataset. We implement a SimCLR-inspired model with a ResNet-50 backbone and evaluate multiple contrastive loss functions, augmentation strategies, and hyperparameters. Using prototypical classification and a 6.25 % training subset, the NT-Xent–based model achieves an average accuracy of 73.6 %. t-SNE and IntCAM analyses show that the model learns partially separated embeddings but struggles to consistently focus on tumor regions. The results highlight both the promise of CRL for low-label medical imaging and key limitations related to augmentation, backbone choice, and training scale.

Keywords Brain Tumor Classification · Self-supervised learning · Prototypical Contrastive Learning

1 Introduction

Precise identification of brain tumors is essential in order to treat them effectively, yet automated analysis of Magnetic Resonance Images (MRI) remains a significant challenge to this day. Variability in tumor shapes and sizes makes them difficult to detect, and the lack of high-resolution, well-annotated datasets often affects the reliability of machine-learning models. Although datasets like BraTS or Figshare have contributed substantially to the field, their limited amount of tumor types and reliance on pre-processed data often do not fully reflect real-world imaging [1] [2]. To address these gaps in the datasets, the Brain tumor Image Segmentation and Classification (BRISC) dataset contains a broader set of expert-annotated MRI images. Containing 6000 images, BRISC offers a representative foundation for developing and evaluating tumor-classification models.

With the dataset follows extensive research on different architectures used for either classification or segmentation. We begin this project by replicating their results, which motivates a full reconstruction of the model architecture. Focusing on classification, we adopt the metrics and evaluation methods from the BRISC paper. Throughout this work, we investigate Contrastive Representation Learning (CRL), comparing the performance of different contrastive loss functions, augmentation strategies, and hyperparameters. Additionally, visualization methods are used as part of the results to investigate model behavior. CRL is particularly appealing for medical imaging, where labeled data is expensive and time-consuming to produce but unlabeled MRIs are abundant, making it an interesting direction for learning representations that generalize across tumor types and MRI variations. The model is evaluated through the use of prototype-based classification, which fits with our goal of learning separated embedding spaces for tumor representation.

2 Related work

2.1 Self-supervised learning

A typical classification model is trained with both pictures and labels in order to classify the pictures according to their most probable labels. This is called *supervised learning*, since humans are required to label the pictures in the supplied dataset. This labeled data can be costly to produce and therefore often in limited supply. This means that labeled classification models are often effective for narrow tasks, but struggle to build broad and flexible knowledge representations.

LeCun & Misra from Meta AI present the concept of *self-supervised learning* (SSL) as the "dark matter of intelligence", since it is pervasive but invisible [3]. In principle, SSL mirrors the way humans learn by observing, predicting, and forming internal models of pictures. It achieves this by generating its own learning signals from the data itself without having to rely on labels provided by humans. This makes the concept of SSL beneficial, since it can scale to huge amounts of unlabeled data with rich representations that are relevant for other tasks than the task used during pretraining.

2.2 Contrastive Representation Learning

Contrastive Representation Learning is a particular modeling approach that allows for the integration of SSL. According to Le-Khac et al. a CRL model "learns by comparing among different samples" rather than learning from data presented one at a time [4, p. 3].

An illustration of the architecture used for CRLs can be seen in figure 1. Here, one or more encoders map a query and *positive* (similar) and *negative* (dissimilar) pairs of images to embeddings. The *positive* pairs are typically different augmentations of the same image, while the *negative* pairs are different images. Then, the contrastive loss function pulls the query closer to positives and pushes it away from negatives. In the end, it aims to learn an embedding-space where positive samples are clustered together and negative ones are separated. However, one of the disadvantages with this approach is that only determining the distances between the positives can result in all embedding converging towards the same point creating a constant function.

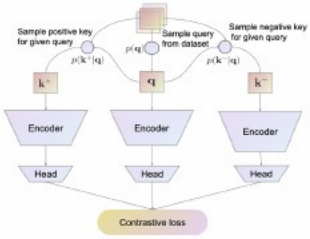

Figure 1: An overview of the CRL framework, showing how it works with positive and negative examples [4].

2.3 Prototypical Contrastive Learning

Li et al. present Prototypical Contrastive Learning (PCL) as "an unsupervised representation learning method that bridges contrastive learning with clustering" [5, p. 1]. If a contrastive learning network is implemented using 'instance-wise' comparisons, they will often fail to capture a higher level semantic structure, as every instance will be pushed apart. So in PCL *prototypes* are introduced, which are 'cluster centroids' obtained via k-means over the learned embeddings. The embeddings are encouraged to move toward the prototype of their assigned cluster and away from other prototypes.

3 Methods

3.1 Dataset & preprocessing

For our project, we use the BRain tumor Image Segmentation and Classification (BRISC) dataset published in 2025. This is a comprehensive dataset, consisting of MRI scans of three classes of tumor (*glioma*, *meningioma*, and *pituitary*) and a *no-tumor* class. The entire dataset contains 6000 images, split into 5000 train and 1000 test images[1].

The dataset is made as a response to the lack of high-quality, balanced, and diverse datasets, which is important in critical fields like medical image analysis. The images are T1-weighted, high-quality images that have been annotated by certified radiologists and physicians. Additionally, the images are also categorized across three different axis, namely the *axial*, *sagittal* and *coronal* planes (figure 2. The dimensions of the images differ, spanning both rectangular and quadratic images, adding to the variation of the dataset.

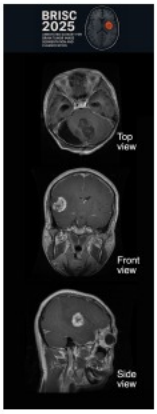

Figure 2: *Glioma* tumor from three angles.

Given the thoroughness of the dataset as explained in the accompanying article, there was not much to find. We identified the issue that not many hospitals would have the resources to professionally annotate that many images. This led us down an experimental path to investigate how one could

[1] https://www.kaggle.com/datasets/briscdataset/brisc2025/data

achieve a good evaluation using less labeled data. Hypothetically, this could reduce the amount of resources any hospital would have to use in order to implement a model of their own.

3.2 Model evaluation

Our primary form of evaluation follows the evaluation metrics for classification used in BRISC dataset paper [6]. The four metrics are *Accuracy*, *Precision*, *Recall*, and *F1-Score*.

With these metrics, we can directly compare to the BRISC paper. The paper states that they trained each model thrice and took the average as the result. We have not done this for our baseline model, but we have done it five times for the final version of our model in section 4.5. During our report, we refer to *accuracy*, because we are interested in how our data classifies on unseen data.

3.3 Baseline CNN model

Our first initial experiment was to reconstruct one of the models from the article as our baseline model. This partially worked as a test of the dataset, and also as a test of how well a relatively standard model could perform without any fine-tuning.

This resulted in a quick setup of a *Convolutional Neural Network (CNN)* with a *ResNet-50 encoder* pretrained on the *ImageNet dataset*. We used an *Adam optimizer* with only a subset (10%) of the data for quicker processing. From this test we received an accuracy of 98.2 %, which is exactly the same as the weighted average from the BRISC paper [6]. With this being the first try without any hyperparameter tuning or finetuning the model, we were curious to how the dataset would perform with an entirely different architecture.

3.4 Model architecture

The model we developed has a *self-supervised CRL* architecture inspired by *SimCLR*. Generally, it consists of three main components; the ResNet-50 backbone, an embedding head, and a projection head. The backbone serves as a feature extractor and provides a high-dimensional feature vector for each MRI. This vector is passed through the embedding head, producing a 512-dimensional normalized embedding that forms the basis of the representation space. Additionally, the model includes a projection head that maps encoder features into a 128-dimensional embedding space, which is the space optimized by the contrastive loss. After training, the backbone and embedding head act as the feature encoder. During prototypical evaluation, each embedding is compared to their corresponding class prototype (the mean embedding per class), enabling classification without training a classifier.

3.4.1 Loss functions

During our project, we came across different types of *contrastive loss functions*. Our baseline use cross-entropy, and we also use *Supervised Contrastive Learning (SupCon)* as a bridge between the baseline CNN and our final model [7]. As the name might imply, it uses labels, but in a CRL setting, where embeddings from the same class are pulled towards each other, and embeddings from different classes are pushed apart.

We mainly use the *Normalized Temperature Cross-Entropy (NT-Xent)* loss function. It is unsupervised, meaning it also compares images in the batch, but uses cosine similarity instead of classes to identify positive (augmented versions of images) and negative (completely different images) pairs based on their embeddings.

3.5 Transfer learning

The model architecture relies on a ResNet-50 backbone pretrained on the ImageNet architecture. By using a pretrained encoder instead of training one from scratch, the model is better at generalization. On top of this pretrained encoder, a decoder is added, trained on the BRISC dataset. The pretrained encoder provides initialized weights that, during decoder training, are customized to fit the BRISC dataset. During the training of the decoder, the entire encoder is frozen, although fine-tuning the model requires unfreezing a variable amount of layers in the encoder, to adapt to more specialized tasks.

3.6 Temperature

When conducting CRL, temperature is an addition to the usual hyperparameter-family. This hyperparameter controls the strength of the penalties on the hard negative samples. A lower temperature (~ 0.07) penalizes the hard negative values more, leading to a more uniform embedding space, whereas a higher temperature (~ 0.2) is less sensitive to hard negative samples, making the embedding space more segmented [8].

3.7 Data augmentation

As previously mentioned, the model is partially inspired by the SimCLR framework, which means that data augmentation is crucial for improving the model. Generally, it is important to use data augmentation in CRL in order to create positive pairs. SimCLR highlights *random cropping*, *random color distortion*, and *random Gaussian blur* as their chosen augmentations [9].

Given a different dataset and use case, the use of data augmentation in this project is elaborated on by utilizing different augmentation families, that are customized to the case. The families can be observed in table 1

Family name	Augmentations	What it tests
basic	RandomResizedCrop(0.7–1.0), HorizontalFlip, Rotation($\pm 10°$)	Mild geometric variation
intensity	`basic`, ColorJitter(brightness ±0.1, contrast ±0.1)	Adds photometric variation; brightness/contrast drift and intensity invariance.
medical	`basic`, RandomApply(ColorJitter), RandomApply(Gaussian Blur)	Medical-aware augmentations that mimics scanner variability without distoriting anatomic representation
strong_geom	RandomResizedCrop(0.6–1.0), HorizontalFlip, Rotation($\pm 30°$)	More aggressive version of `basic` family

Table 1: Augmentation families and their purpose.

3.8 Visualization methods

3.8.1 t-SNE

t-Distributed Stochastic Neighboring Entities (tSNE) is a method for dimensionality reduction for easier visualization of the embedding space of a model. It provides a rough idea of the overall topological tendencies, making it easier to analyze the discrimination between classes. One of the perks of this method is that if two high-dimensional embeddings are close, they are represented as close in the tSNE model, even though there are only two or three dimensions [10].

3.8.2 Interactive-CAM (IntCAM)

This explainability method extends *Grad-CAM* to contrastive learning by incorporating interactions between paired images. Because standard Grad-CAM produces explanations for a single image, it does not reflect how two views jointly influence the contrastive objective. IntCAM addresses this by reducing each feature map to a scalar (we use mean reduction) and then computing an element-wise product between the two activation vectors. This highlights features that are strongly activated in both images while suppressing those activated in only one, enabling pair-aware visualization suited to CRL [11].

4 Results

4.1 SupCon vs. NT-Xent

To investigate the effect of supervision (labels) in contrastive learning, we conduct experiments using both NT-Xent and SupCon loss. The accuracy achieved by these experiments can be seen in figure 3. Comparing the two allows us to evaluate whether supervision leads to more discriminative embeddings for downstream prototypical classification. During our initial testing with a subset of just 6.2% (308 images for training), we achieved an accuracy of 92.2% using SupCon and 74.6% using NT-Xent, illustrating quite a difference.

Additionally, we tried to reduce the size of the subset to evaluate the effect in the case of an extremely small amount of data ($\sim 0.8\%$), and saw that the accuracy of SupCon gradually decreases proportionally to the size of the subset, with an accuracy of only 68.1%. This emphasizes that not only the labels, but also the amount

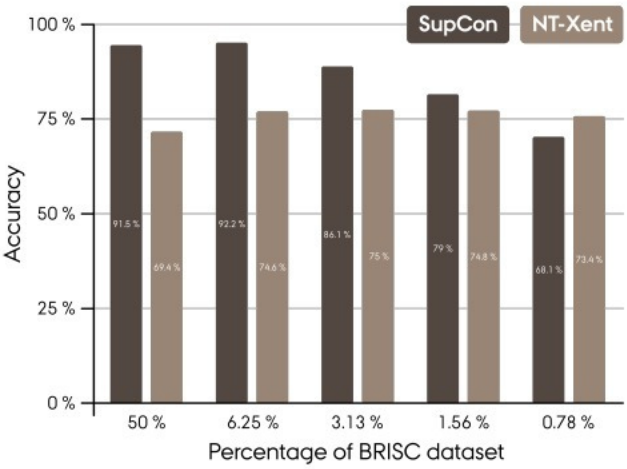

Figure 3: Comparisons for the SupCon and NT-Xent contrastive loss functions with increasingly small subsets of the BRISC dataset for training.

of labels, are significant to achieve a higher accuracy. In contrast, NT-Xent did not show a significant drop in accuracy, still maintaining 73.4% accuracy with a $\sim 0.8\%$ subset size. Thus, we can conclude that because NT-Xent is not reliant on labels, it can uphold a relatively high accuracy even with extremely small dataset sizes.

From this point forward, all of our experiments are made with the NT-Xent loss function only. Likewise, our subset size is fixed to 6.2%. As the accuracy drop is relatively small, this is a compromise between decreasing training time and maintaining accuracy.

4.2 Hyperparameter tuning

4.2.1 Temperature

Up until this point, our model was trained with a low temerature value (0.07), yielding accuracies in the mid-70s. However, when increasing the temperature to a more moderate value (0.25), we observed that the accuracy increased to a temporary all-time high of 77.6%, indicating that a more strictly divided embedding space is preferable. However, increasing the temperature to 0.5 resulted in an accuracy of 75.7%, indicating that it could also become too strict.

From this point forward, all of our experiments have a temperature of 0.25 unless stated otherwise.

4.2.2 Learning rate & epochs

As an extension of the experiments above, we were curious to see what other hyperparameters could change. Up until this point, the number of epochs was intentionally small to decrease training time. However, changes in temperature increased our training loss from ~ 0.005 to > 1. As we were unable to increase the batch size (which we discuss in section 5.1), we determined that the learning rate and the number of epochs were the place to begin.

In the experiments, the epochs range from 60-1000, and the learning rate ranges from $1\mathrm{e}-4$ to $1\mathrm{e}-7$. As a sub-experiment, one can observe table 2 and easily conclude that a lower learning rate results in a higher loss, which in turn would point to a high learning rate to minimize loss. However, the loss curves in figure 4 indicate that the higher learning rate is not as efficient as the lower ones.

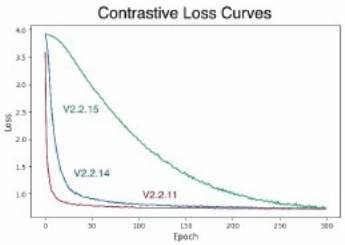

Figure 4: Loss curves for the versions noted in table 2

Version	Learning rate	Epochs	Loss
V2.2.11	$1\mathrm{e}-4$	300	0.6998
V2.2.14	$1\mathrm{e}-5$	300	0.7275
V2.2.15	$1\mathrm{e}-6$	300	1.0332

Table 2: The relation between learning rate and loss

As the loss was higher, but the learning rate was more efficient, we tried to increase the number of epochs to 500 and 1000, respectively. This resulted in lower losses (0.88 & 0.78), but more importantly, our best accuracy thus far. Training for 500 epochs yielded an accuracy of 78.7%, achieved with an efficient loss curve.

This means that for the next experiment, we fixed our learning rate at $1\mathrm{e}-6$. As there were only 0.2% difference in accuracy between the model trained on 300 and 500 epochs, respectively, we fix the next model on 300 epochs for faster training times.

4.3 Augmentation families

As mentioned earlier, when using the SimCLR framework, data augmentation is crucial for improving the model. We have used four different augmentation families (*basic*, *intensity*, *medical*, and *strong_geom*), as shown in table 1. We started with our best performing model, V2.2.15, and used the different augmentation families with this, without changing any other parameters. Starting with an accuracy of 78.7%, we began testing with strong geometry, which gave a much lower accuracy of 72%. This indicates that a stronger geometric data augmentation will not increase the accuracy in our case.

From what can be seen in table 3, the different augmentation families do not achieve better accuracy. All augmentation families achieved an accuracy between 72% and 73%, except for basic.

Version	Augmentation	Accuracy
V2.2.15	basic	78.7%
V2.2.15.3	strong_geom	72%
V2.2.15.4	intensity	72.8%
V2.2.15.8	medical	72.9%

Table 3: Accuracy of the same model with differing augmentations

Additionally, we tweaked some parameters with the augmentation families. With the intensity family, we changed brightness and contrast. We found that changing both brightness and contrast to 0.5 gave a higher accuracy of 76.2%, compared to 72.8% with the default intensity parameters. Changing the parameters any further resulted in lower accuracies again. With the medical augmentation family, we changed the p-value of a random gaussian blur from the default 0.3 to 0.6, which resulted in a higher accuracy of 77.4%. Although this was a much higher accuracy than the default values of the medical family, it still did not exceed the accuracy of using basic augmentation. This means that for the following experiments we will continue using the basic augmentation family.

4.4 Visual analysis

4.4.1 t-SNE

In figure 5, two embedding spaces can be observed. This setup is inspired by Wang et al. to see if the embedding space discriminates between classes across the use of different temperatures [8]. The variation in temperature makes for entirely different embedding spaces, but even though the embeddings are more evenly distributed with a low temperature, distinct groups are still observable. The prototype embedding can be observed distinctly in the center of each embedding class. We do notice a partial overlap in embeddings, which matches with the fact that our model does not have perfect accuracy. This could imply that images from different classes might have shared features. A cause for this might be that the model trains on images from three different axis, and therefore these are developing shared features, because the images are more similar in structure. We will address a potential solution to this in the discussion.

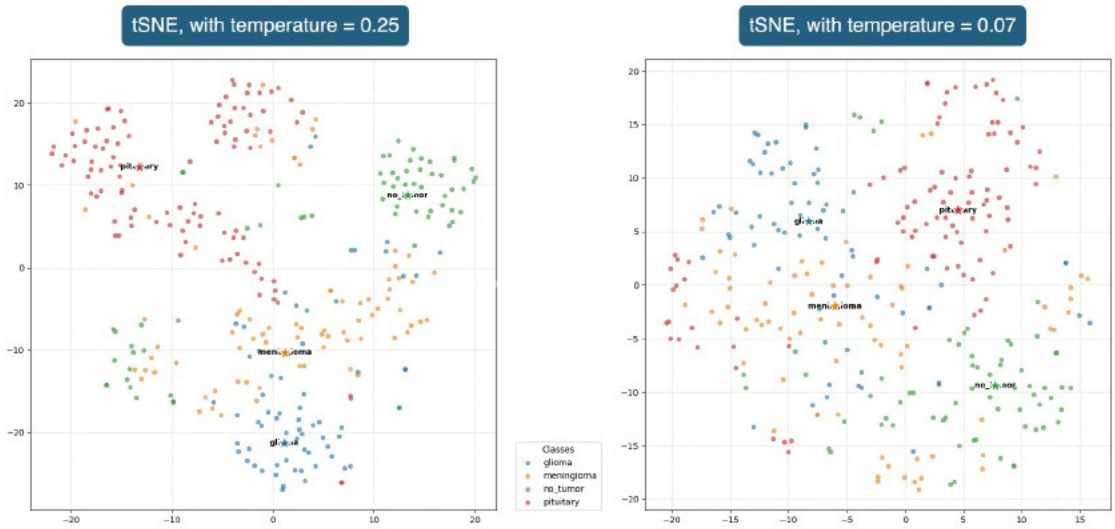

Figure 5: Embedding spaces with different temperatures

4.4.2 Interactive-CAM

In figure 6, an image of each class is shown with the intCAM method applied. We notice that across all four classes, the model has a difficult time accurately focusing it's attention on the tumor, and instead focuses more on the brain itself. In several examples, there seem to be a slight overlap between the red part of the heatmap and the tumor, but the heatmap seems to focus on the center of the brain itself instead of the region with the tumor. This might be a result of SimCLR being unsupervised, and therefore the model learns more global structures than specific ones, such as a tumor. Additionally, if the model is trained with heavy data augmentation, it could mistake the tumor for intentional occlusion, and simply disregard it as noise when trying to detect the brain.

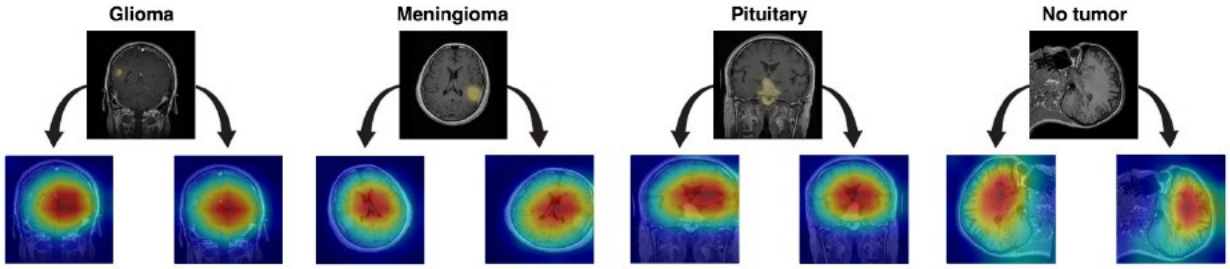

Figure 6: The original MRI image with the tumor located, along with two augmented versions using intCAM

4.5 Robustness of the final model

As a final reference to the BRISC paper, and to assess the stability of our final model configuration, we trained and evaluated the complete pipeline five separate times using identical hyperparameters. The reported values seen in table 4 reflect the mean performance across all three iterations. Averaging several model iterations provide a more reliable estimate of the model's true behavior and reduces the influence on the individual run of a model.

In terms of specifications of the final version, it is implemented using an ADAM optimizer, $1\mathrm{e}-5$ learning rate, 100 epochs, 0.07 temperature, 32 batch size, a 6.25%, an NT-Xent loss function, and a basic augmentation family. This setup reflect a combination of the optimal results from each experiment, combined in a conclusive model. The mean results can be observed below in table 4.

Model iteration	Precision	Recall	F1-Score	Accuracy
1	0.728	0.759	0.729	72.6 %
2	0.732	0.763	0.729	73.4 %
3	0.768	0.780	0.766	75.8 %
4	0.752	0.777	0.748	74.8 %
5	0.718	0.746	0.723	71.5 %
Average	0.739	0.765	0.739	73.62 %

Table 4: Augmentation families and their purpose.

Based on the findings of this table, we conclude that the results may vary with around $\pm 2\%$ when hyperparameters are fixed. Because the composition of CRL relies upon data augmentation, this introduces randomness to the model, which seemingly influences the overall accuracy. Therefore, when we experience high accuracy values above, it could also just be a 'lucky' training session. This could be why the model that otherwise was our best performing one through our project, only returns the average seen in table 4.

5 Discussion

In this section, we discuss potential ways of improving the model, as well as how we could improve the methods we already use.

5.1 A larger batch size

According to Chen et al., CRL should have large batch sizes (> 256) because it increases the ratio of negative pairs to one positive pair, making the model more robust [12]. Ideally, we would like to conduct all of the experiments with a batch size of 1024, which would increase the amount of negative pairs by a multiple of 33. This would increase the diversification of the negative samples, emphasizing stronger separations in the embedding space, which we could analyze using t-SNE.

Unfortunately, a byproduct of using Google Colab as our environment meant that we had limited VRAM during the entire project. As a result of this, our batch size was limited to 32 during our project. Following the work of Chen et al., we assess that a larger batch size would increase the accuracy, and our model would become better at identifying the tumor instead of the brain itself [12]. We could analyze the behavior of the model at several batch sizes and epochs using intCAM to assess whether it has the correct focus.

5.2 More specific prototypes

During our project, we have had four prototypes, each a mean embedding of the entire class. By doing this, we assume that all samples form one coherent cluster of embeddings. However, given that there are images of tumors from three anatomical axis in each class (and as the t-SNE visual might suggest), our classes might produce more than just one cluster. By only having four prototypes, we might obscure underlying structures in the embedding space.

We could accommodate the structures by instead producing a prototype per anatomical axis in each class instead, producing 12 prototypes instead of four. In order for all 12 prototypes to be representative, we would increase the subset used. Ultimately, this could switch the model's attention from the entire brain to the tumor, as well as increase model robustness.

5.3 Reflection on default choices

Due to our initial experiments revolving around the architecture of the model, we initially opted for what we had learned to be "good default choices". What this means for our project is that we have not experimented with these in a way we deem satisfactory. Therefore, this section of the discussion is dedicated to considerations regarding optimizers, backbone and pretraining, and learning rate approach.

5.3.1 A generalized setup for a specialized task

Our backbone consists of ResNet-50 pretrained on ImageNet, which makes it suitable for general purposes. However, our task is specialized, and thus the use of this backbone and pretraining is debatable. First of all, the textures, colors, and other low-level features learned in the pretraining might distort what the model looks for in MRI's, as the anatomical structures differ from natural images. Looking at mid- and high-level features, the object parts and semantics derived in the pretraining could interfere with the detail oriented distinguishing we need to properly and securely identify tumors. If the model has a wide focal point of view, it has difficulty catching crucial details that could determine a tumor. This ultimately restricts the quality of the learned embeddings, as the model uses filters not customized for medical use.

Instead of this general backbone, we consider three alternatives to our concept. These generally follows the ideas of Zhang et al., which focuses on a general dataset for medical imaging [13].

As a *first alternative*, we could use a grayscale version of ImageNet. This could nudge the features in the direction of medical imaging by eliminating the model's concept of colors. *The second alternative* could be to pretrain on a dataset made for this exact purpose. An example of this could be the CPMID dataset proposed by Zhang et al., which is customized to be general within the medical field [13]. *The third alternative* could be to train the backbone from scratch ourselves. This way, we could specialize it completely for tumor identification instead of general medical use. We would completely ignore all of the redundant features from the natural images in ImageNet, making the model focus only on the specific behavior we wish. However, constructing a robust, evenly distributed dataset is a project in itself. Although we have considered it, we deem it out of scope for our project.

5.3.2 SGD + Momentum instead of Adam

Another factor that may have influenced our results is the choice of optimizer. We selected Adam due to its robustness and ease of use, particularly when training with a fixed learning rate. While Adam is a strong default choice, several studies indicate that SGD with momentum can produce better generalization and more stable feature representations. However, SGD requires more careful tuning of both the learning rate and its schedule, which placed it outside the scope of our implementation. Exploring SGD with momentum in combination with an appropriate scheduling strategy may have led to stronger and more discriminative embeddings in our contrastive setup.

5.3.3 A cyclical learning rate

A final limitation we wish to address is the use of a fixed learning rate. An alternative would be a cyclical learning rate schedule, which is known to benefit contrastive learning models by stabilizing early optimization and improving representation quality. For example, the SimCLR paper employs a 10-epoch linear warmup followed by cosine decay without restarts [9].

Although we did not have time to implement it, our training setup could have incorporated a cyclical schedule, either with Adam or SGD with momentum. This would allow the model to begin with a large learning rate, encouraging broad exploration of the representation space, and then gradually reduce it during later epochs to refine the embeddings. Additionally, experimenting with linear warmup could further stabilize early contrastive training and may have improved the quality of our learned features.

References

[1] Bjoern H Menze, Andras Jakab, Stefan Bauer, Jayashree Kalpathy-Cramer, Keyvan Farahani, Justin Kirby, Yuliya Burren, Nicole Porz, Johannes Slotboom, Roland Wiest, et al. The multimodal brain tumor image segmentation benchmark (brats). *IEEE transactions on medical imaging*, 34(10):1993–2024, 2014.

[2] Jun Cheng. brain tumor dataset. 4 2017. doi:10.6084/m9.figshare.1512427.v8. URL https://figshare.com/articles/dataset/brain_tumor_dataset/1512427.

[3] Yann LeCun and Ishan Misra. Self-supervised learning: The dark matter of intelligence. https://ai.meta.com/blog/self-supervised-learning-the-dark-matter-of-intelligence/, March 2021. Meta AI Blog.

[4] Phuc H. Le-Khac, Graham Healy, and Alan F. Smeaton. Contrastive representation learning: A framework and review. *arXiv preprint arXiv:2010.05113*, 2020. doi:10.48550/arXiv.2010.05113. URL https://arxiv.org/abs/2010.05113. Version v2 (27 Oct 2020).

[5] Junnan Li, Pan Zhou, Caiming Xiong, and Steven C. H. Hoi. Prototypical contrastive learning of unsupervised representations. *arXiv preprint arXiv:2005.04966*, 2020. doi:10.48550/arXiv.2005.04966. URL https://arxiv.org/abs/2005.04966. arXiv:2005.04966, version v5 (30 Mar 2021).

[6] Amirreza Fateh, Yasin Rezvani, Sara Moayedi, Sadjad Rezvani, Fatemeh Fateh, Mansoor Fateh, and Vahid Abolghasemi. Brisc: Annotated dataset for brain tumor segmentation and classification, 2025. URL https://arxiv.org/abs/2506.14318.

[7] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning. *Advances in neural information processing systems*, 33: 18661–18673, 2020.

[8] Feng Wang and Huaping Liu. Understanding the behaviour of contrastive loss. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2495–2504, 2021.

[9] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey E. Hinton. A simple framework for contrastive learning of visual representations. *CoRR*, abs/2002.05709, 2020. URL https://arxiv.org/abs/2002.05709.

[10] Harshvardhan Gupta. Using T-SNE to Visualise how your Deep Model thinks — medium.com. https://medium.com/buzzrobot/using-t-sne-to-visualise-how-your-deep-model-thinks-4ba6da0c63a0. 2017. [Accessed 26-11-2025].

[11] Fawaz Sammani, Boris Joukovsky, and Nikos Deligiannis. Visualizing and understanding contrastive learning. *IEEE Transactions on Image Processing*, 33:541–555, 2023.

[12] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PmLR, 2020.

[13] Rongguo Zhang, Chenhao Pei, Ji Shi, and Shaokang Wang. Construction and validation of a general medical image dataset for pretraining. *Journal of Imaging Informatics in Medicine*, 38(2):1051–1061, 2025.